MasterIntegration-Patterns

Oct 13, 2026 · @Soumyabrata Manna

1. Introduction: Why Patterns Matter

Integration patterns are named, proven solutions to recurring problems in connecting applications, so a team can choose a design by its trade-offs instead of inventing one each time. Most come from Hohpe and Woolf's *Enterprise Integration Patterns*, extended by API, streaming and cloud-era practice.

Every pattern below follows the same shape: the problem it solves, a diagram, a use case, sample code and trade-offs. The code is short and illustrative. It leaves out authentication, logging and configuration, which production code needs.

Part	Patterns	Core question
2. Topology	Point-to-point, hub-and-spoke, ESB, API-led connectivity	How are systems wired together?
3. Messaging	Message queue, publish/subscribe, event streaming	How do messages travel?
4. Interaction	Request/reply, fire-and-forget, asynchronous request-reply	Does the caller wait?
5. Data	Batch/ETL, change data capture, data synchronization, canonical model	How does data move and stay consistent?
6. Routing and transformation	Content-based router, splitter, aggregator, scatter-gather	How are messages directed and reshaped?
7. Resilience	Retry, circuit breaker, dead-letter queue, idempotent receiver, saga	What happens when something fails?

Parts 8 to 12 add integration styles, channels and endpoints, message construction, further routing and transformation patterns, and legacy migration patterns. Part 13 maps the patterns to Boomi, and part 14 is the selection guide.

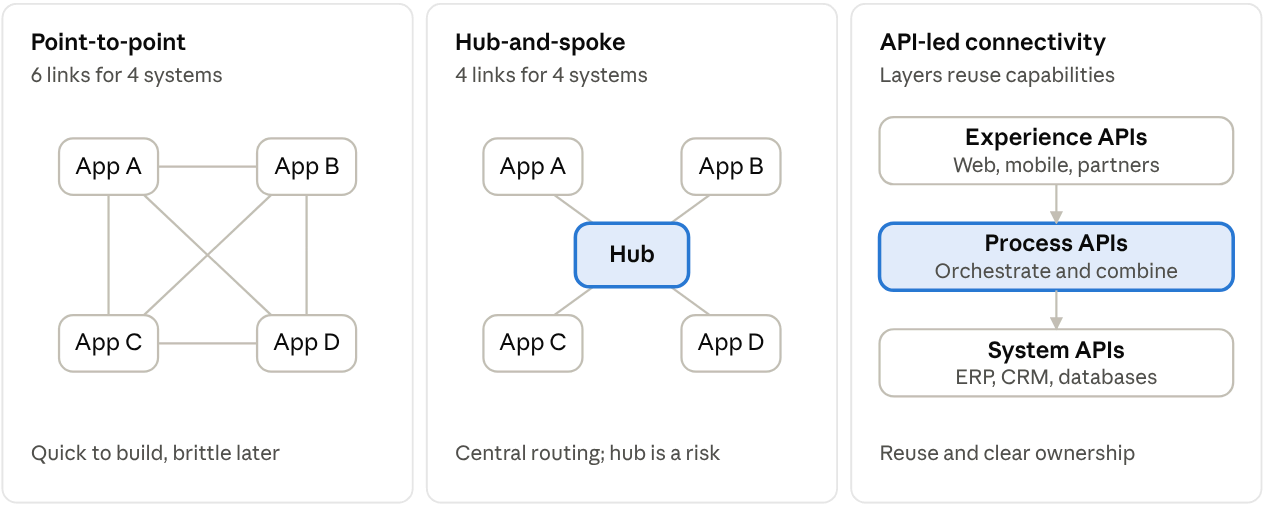
Four questions usually decide which pattern fits:

1. **Timing**: must the caller wait for the answer (synchronous) or not (asynchronous)?
2. **Volume and latency**: a few records per second in real time, or millions per night?
3. **Coupling**: can sender and receiver be up, changed and scaled independently?
4. **Consistency**: is eventual consistency acceptable, or must a change succeed everywhere or nowhere?

2. Topology Patterns

Topology patterns decide how many connections a landscape needs and where the integration logic lives.

Direct links multiply; a hub or API layers keep wiring manageable



topology patterns · 3 panels

Point-to-point needs a link per pair of systems, a hub concentrates routing in one place, and API-led layers separate the consumers of data from its owners.

2.1 Point-to-point

One system calls another directly, with no middleware in between. **Use case:** a web shop sends each paid order straight to the ERP.

```
import requests

def push_order_to_erp(order: dict) -> None:
    response = requests.post(
        "https://erp.example.com/api/orders",
        json={"id": order["id"], "total": order["total"], "lines":
order["lines"]},
        timeout=10,
    )
    response.raise_for_status()
```

Trade-offs: fastest to build and easy to debug with two systems. With n systems the worst case is n × (n − 1) / 2 links, and every change ripples to each partner.

2.2 Hub-and-spoke

Every system connects only to a central broker, which routes and translates messages. **Use case:** a retailer's POS, ERP, warehouse and CRM each talk to one integration platform and never to each other.

```
ROUTES = {
    "order.created": ["erp", "wms"],
    "customer.updated": ["crm", "erp"],
}

def hub_dispatch(message: dict, adapters: dict) -> None:
    for target in ROUTES.get(message["type"], []):
        adapters[target].send(message)  # each adapter translates to the
target's format
```

Trade-offs: links grow linearly and rules live in one place. The hub becomes a bottleneck and a single point of failure unless it is clustered.

2.3 Enterprise Service Bus (ESB)

A distributed hub that adds protocol mediation, transformation, routing and monitoring across many runtimes. **Use case:** a bank connects a mainframe on MQ, SOAP partner services and modern REST APIs. The Apache Camel route below reads JMS, translates and calls REST.

```
from("jms:queue:orders.in")
    .routeId("esb-orders")
    .unmarshal().json(JsonLibrary.Jackson, Order.class)
    .to("bean:orderTranslator")
    .to("http://erp.example.com/api/orders");
```

Trade-offs: strong mediation and governance. Heavy, centrally owned bus platforms can slow delivery, so many teams now favor lighter runtimes with API-led design.

2.4 API-led connectivity

Capabilities are exposed in three layers: system APIs wrap back-end systems, process APIs orchestrate them, and experience APIs serve each channel. **Use case:** a customer-summary process API combines CRM, orders and billing system APIs, and both the mobile app and the partner portal reuse it.

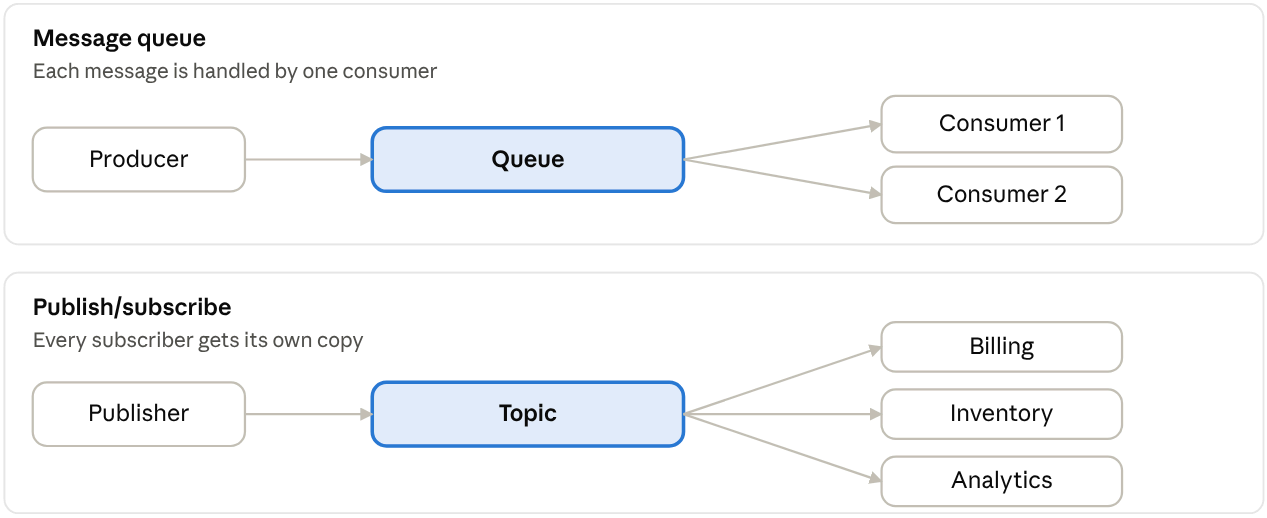
```
paths:
  /customers/{id}/summary:
    get:
      summary: Customer summary (process API)
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: string }
      responses:
        "200":
          description: Combined profile, open orders and balance
```

Trade-offs: high reuse and clear ownership, at the cost of more layers to design, version and secure.

3. Messaging Patterns

Messaging patterns put a broker between sender and receiver so the two no longer need to be available at the same moment.

A queue delivers each message once; a topic delivers it to every subscriber



queue vs publish/subscribe · 2 flows

In a queue, consumers compete for messages and share the load. In publish/subscribe, each subscriber receives every message.

3.1 Message queue

A producer puts messages on a queue and one consumer takes each message later. **Use case:** order intake spikes during a sale, and the queue lets the ERP process orders at a steady pace instead of failing under load. The sample uses RabbitMQ with the `pika` library.

```
import json
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
channel = connection.channel()
channel.queue_declare(queue="orders", durable=True)

# Producer
channel.basic_publish(
    exchange="",
    routing_key="orders",
    body=json.dumps({"order_id": "A-1001", "total": 59.90}),
    properties=pika.BasicProperties(delivery_mode=2),  # persist the message
)
```

```
# Consumer (same connection and channel setup as above)
def handle(channel, method, properties, body):
    order = json.loads(body)
    process_order(order)  # your business logic
    channel.basic_ack(delivery_tag=method.delivery_tag)  # ack only after
success

channel.basic_qos(prefetch_count=10)
channel.basic_consume(queue="orders", on_message_callback=handle)
channel.start_consuming()
```

Trade-offs: absorbs spikes and decouples availability. Delivery is usually at-least-once, so consumers must tolerate duplicates (see the idempotent receiver in section 7).

3.2 Publish/subscribe

A publisher sends an event to a topic and every interested subscriber receives a copy, without the publisher knowing who they are. **Use case:** when an order is placed, billing, inventory and analytics each react independently. The sample uses Kafka with `confluent-kafka`. Subscribers that use different `group.id` values each get every message, while consumers sharing one group split the messages like a queue.

```
from confluent_kafka import Producer, Consumer

producer = Producer({"bootstrap.servers": "localhost:9092"})
producer.produce("order-placed", key="A-1001", value='{"order_id": "A-
1001"}')
producer.flush()

consumer = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "billing",  # inventory and analytics would use their own
group ids
    "auto.offset.reset": "earliest",
})
consumer.subscribe(["order-placed"])
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    print("billing received", msg.value())
```

Trade-offs: new subscribers can be added without touching the publisher. Event contracts become shared assets that need versioning, and debugging spans several consumers.

3.3 Event streaming

Events are kept in an ordered, replayable log, and consumers process them continuously, even re-reading history. **Use case:** card payments flow into a topic, a fraud service flags unusual ones in near real time, and the same log can be replayed later to retrain a model. The Kafka Streams sketch below filters large payments into a second topic (serializer configuration omitted).

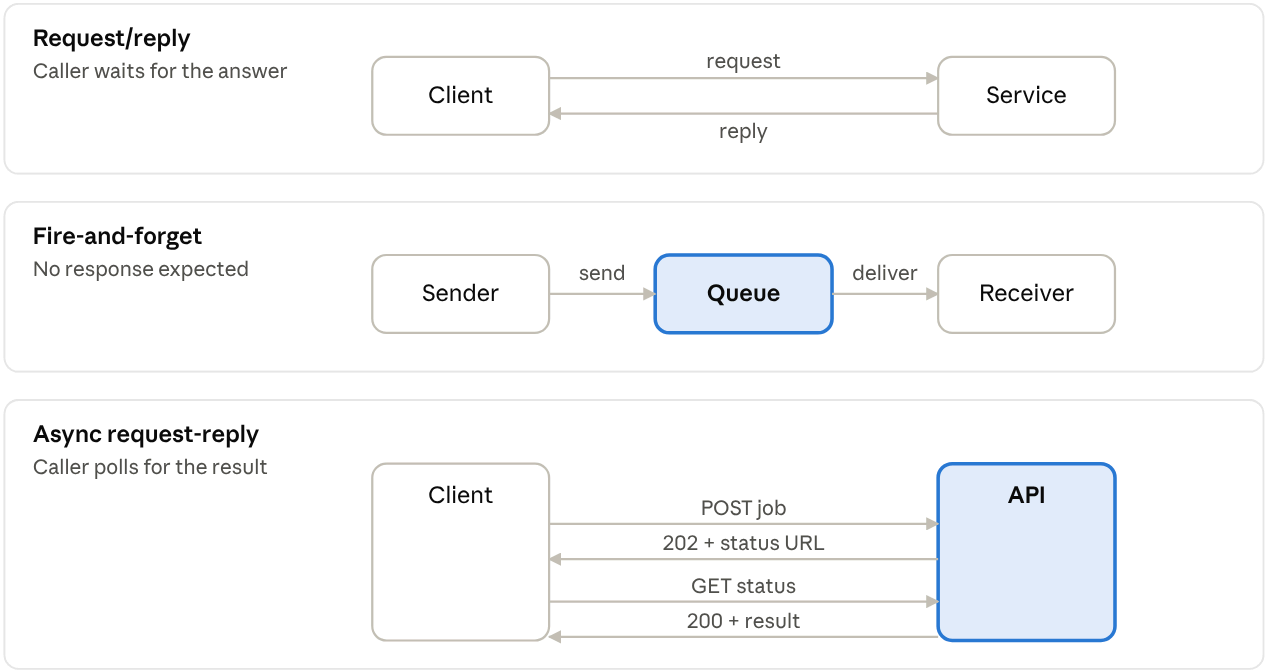
```
StreamsBuilder builder = new StreamsBuilder();
builder.<String, Payment>stream("payments")
       .filter((id, payment) -> payment.amount() > 10_000)
       .to("payments-flagged");
```

Trade-offs: low latency and replay for audit or rebuilds. It needs operational skill with partitions, ordering and retention, and is more than a simple job needs.

4. Interaction Patterns

Interaction patterns define whether the caller waits, and what it gets back.

Only request/reply makes the caller wait; the other two free it at once



interaction patterns · 3 flows

Request/reply blocks until the answer arrives. Fire-and-forget returns as soon as the message is handed off. Asynchronous request-reply returns at once with a ticket, and the client collects the result later.

4.1 Request/reply

The caller sends a request and waits for the response. **Use case:** checkout asks a payment service to authorize a card and needs an immediate yes or no.

```
import requests

def authorize_payment(card_token: str, amount: float) -> bool:
    response = requests.post(
        "https://payments.example.com/v1/authorizations",
        json={"token": card_token, "amount": amount},
        timeout=3,  # never wait forever
    )
    response.raise_for_status()
    return response.json()["approved"]
```

Trade-offs: simple and immediate. The caller is blocked and inherits the callee's availability, so always set timeouts and pair calls with retries and a circuit breaker (section 7).

4.2 Fire-and-forget

The sender hands a message to a broker and moves on without expecting a response. **Use case:** after signup, the app queues a welcome email, and a worker sends it later. The sample uses Amazon SQS through `boto3`.

```
import json
import boto3

sqs = boto3.client("sqs")

def notify_signup(user_id: str) -> None:
    sqs.send_message(
        QueueUrl="https://sqs.us-east-1.amazonaws.com/123456789012/welcome-
emails",
        MessageBody=json.dumps({"user_id": user_id}),
    )
    # return immediately; a worker sends the email later
```

Trade-offs: low latency and a sender that survives receiver outages. The sender never learns about failures, so add monitoring and a dead-letter queue (section 7).

4.3 Asynchronous request-reply

The API accepts the work, answers `202 Accepted` with a status URL, and the client polls (or receives a callback) for the result. **Use case:** a user requests a large report that takes minutes to build.

```
import uuid
from fastapi import FastAPI, BackgroundTasks, HTTPException

app = FastAPI()
jobs: dict[str, dict] = {}  # use a database or cache in production

def run_report(job_id: str) -> None:
    result = build_big_report()  # long-running work
    jobs[job_id] = {"status": "done", "result": result}
```

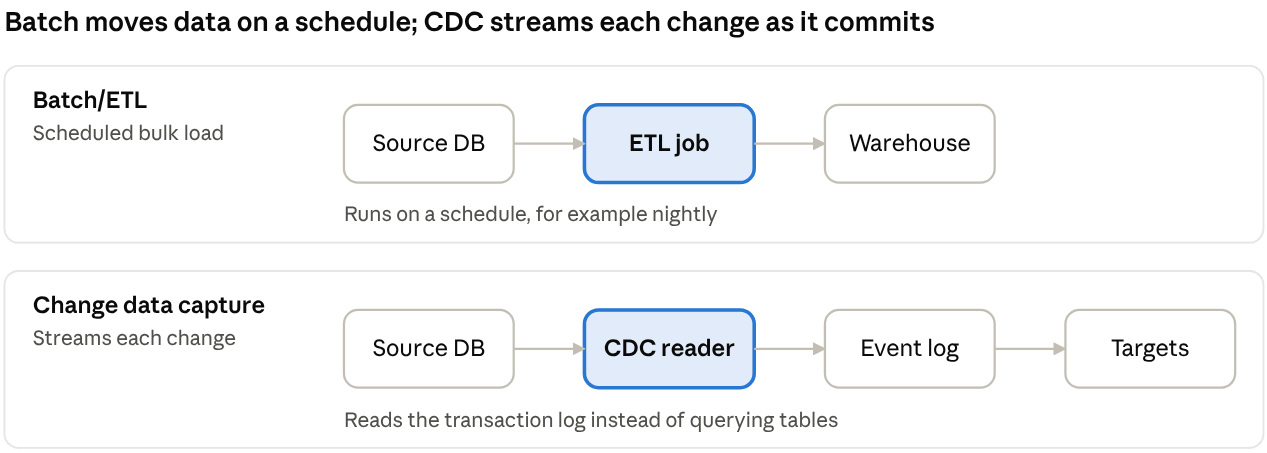
```
@app.post("/reports", status_code=202)
def start_report(background: BackgroundTasks):
    job_id = str(uuid.uuid4())
    jobs[job_id] = {"status": "running"}
    background.add_task(run_report, job_id)
    return {"job_id": job_id, "status_url": f"/reports/{job_id}"}

@app.get("/reports/{job_id}")
def get_report(job_id: str):
    if job_id not in jobs:
        raise HTTPException(status_code=404)
    return jobs[job_id]
```

Trade-offs: avoids gateway timeouts on long work. The client must poll or accept a webhook, and the server must store job state and expire it.

5. Data Patterns

Data patterns cover how data moves between systems and how copies stay consistent.



batch vs change data capture · 2 flows

Batch moves many records at set times, while change data capture (CDC) emits each change shortly after it commits.

5.1 Batch/ETL

A scheduled job extracts data, transforms it and loads it into a target. **Use case:** a nightly load of the day's orders into a data warehouse. The sketch below extracts only rows changed since the last run, using a watermark, and loads them idempotently (PostgreSQL

syntax).

```
def extract_changed_orders(conn, last_run):
    cur = conn.cursor()
    cur.execute(
        "SELECT id, customer_id, total, updated_at "
        "FROM orders WHERE updated_at > %s ORDER BY updated_at",
        (last_run,),
    )
    return cur.fetchall()

def load_to_warehouse(wh_conn, rows):
    cur = wh_conn.cursor()
    cur.executemany(
        "INSERT INTO fact_orders (id, customer_id, total, updated_at) "
        "VALUES (%s, %s, %s, %s) "
        "ON CONFLICT (id) DO UPDATE "
        "SET total = EXCLUDED.total, updated_at = EXCLUDED.updated_at",
        rows,
    )
    wh_conn.commit()
```

Trade-offs: efficient for large volumes and easy to schedule and restart. Data is stale between runs, and a timestamp watermark cannot see deleted rows.

5.2 Change data capture (CDC)

A connector reads the database transaction log and publishes every insert, update and delete as an event. **Use case:** keep a search index and a cache in step with the orders table within seconds. This Debezium connector configuration streams one PostgreSQL table to Kafka.

```
{
  "name": "orders-cdc",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "db.example.com",
    "database.port": "5432",
    "database.user": "cdc_user",
    "database.password": "${file:/secrets/db.properties:password}",
    "database.dbname": "shop",
    "topic.prefix": "shop",
    "table.include.list": "public.orders",
    "plugin.name": "pgoutput"
  }
}
```

A consumer then applies each change event to the target:

```
import json

event = json.loads(message.value())
op = event["payload"]["op"]  # c = create, u = update, d = delete
if op in ("c", "u"):
    upsert_into_target(event["payload"]["after"])
elif op == "d":
    delete_from_target(event["payload"]["before"]["id"])
```

Trade-offs: near real time, captures deletes and puts little query load on the source. It needs access to the database log and operational care for schema changes and connector restarts.

5.3 Data synchronization

Two or more systems keep the same records consistent, one-way or in both directions. **Use case:** a customer's address can be edited in both CRM and billing. The upsert below applies an incoming change only if it is newer than the stored one (last writer wins).

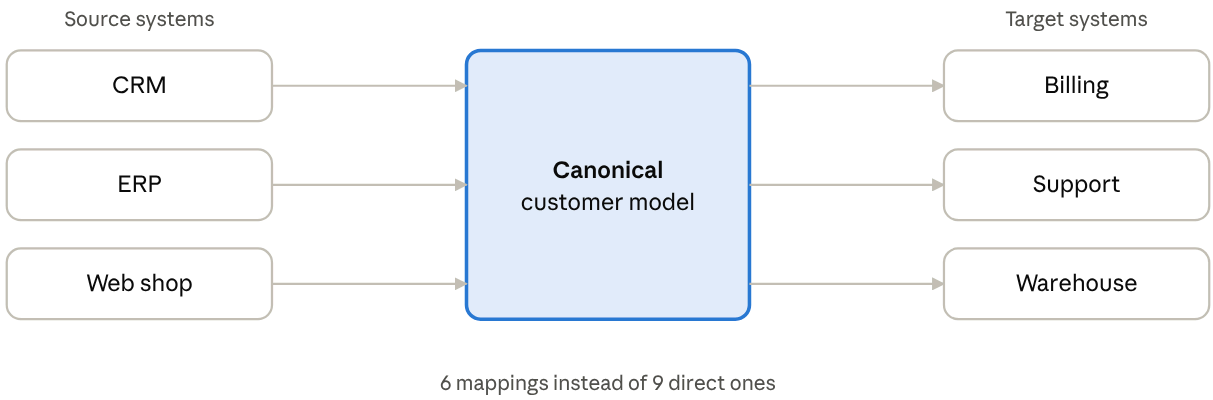
```
INSERT INTO customer (id, email, address, updated_at)
VALUES (:id, :email, :address, :updated_at)
ON CONFLICT (id) DO UPDATE
SET email = EXCLUDED.email,
    address = EXCLUDED.address,
    updated_at = EXCLUDED.updated_at
WHERE customer.updated_at < EXCLUDED.updated_at;  -- last writer wins
```

Trade-offs: simple rule, but last-writer-wins can silently drop an edit. For important fields, name one system of record per field or use a master data hub, and tag each change with its origin so two-way sync does not loop.

5.4 Canonical data model

Every system maps once to a shared, neutral format instead of to every other system. **Use case:** customer data from CRM, ERP and the web shop is mapped to one customer model that billing, support and the warehouse all consume.

A canonical model needs one mapping per system instead of one per pair



canonical data model · 3 sources, 3 targets

Adding a new system means writing one mapping, not one per existing partner.

```
def crm_to_canonical(crm: dict) -> dict:
    return {
        "customerId": crm["AccountId"],
        "fullName": f"{crm['FirstName']} {crm['LastName']}",
        "email": crm["Email"],
    }

def canonical_to_billing(c: dict) -> dict:
    return {"cust_no": c["customerId"], "name": c["fullName"],
"contact_email": c["email"]}
```

Trade-offs: mappings grow linearly and meaning stays consistent. The model needs an owner and versioning, and an over-general model can lose detail that one system needs.

5.5 Data federation

A virtual query layer reads from several sources in place, without copying their data into a new store. **Use case:** finance wants revenue by customer, combining a CRM database with a sales database, without building a pipeline first. In Trino, each source is a catalog, so one SQL statement can join them.

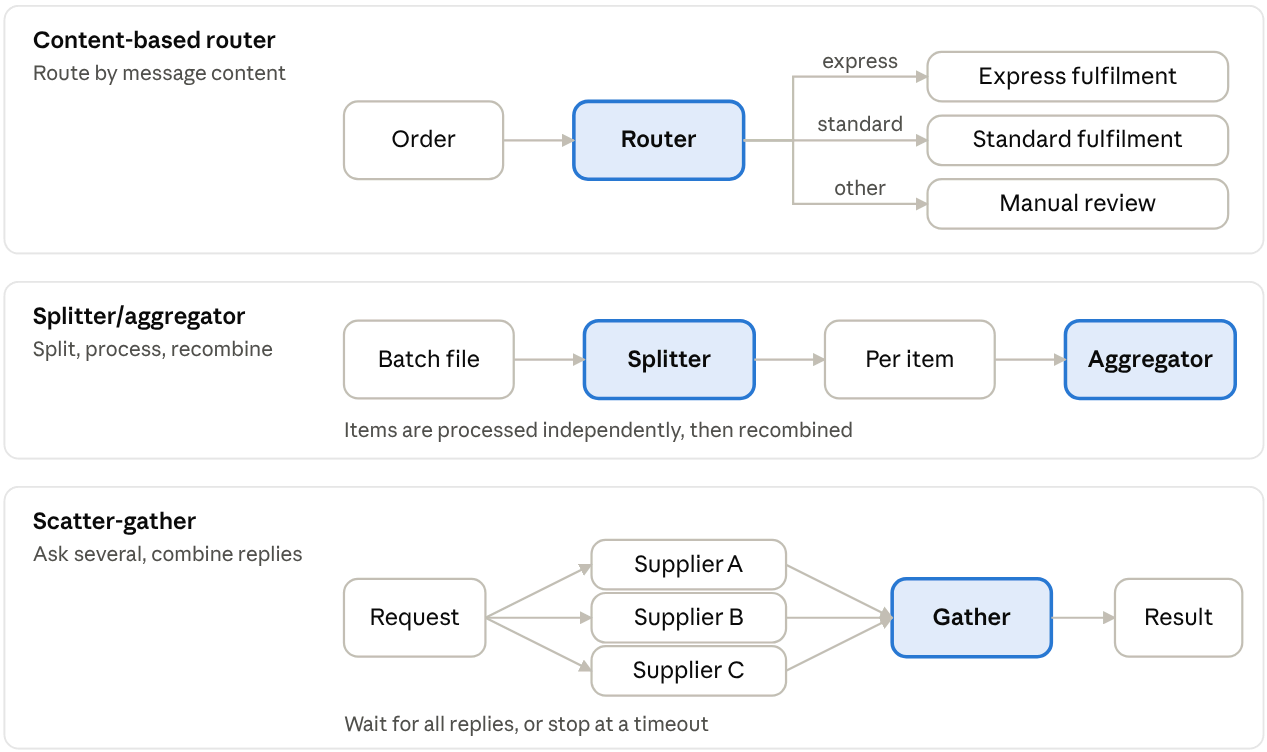
```
SELECT c.name, SUM(o.total) AS revenue
FROM crm.public.customers AS c
JOIN sales.public.orders AS o ON o.customer_id = c.id
GROUP BY c.name
ORDER BY revenue DESC;
```

Trade-offs: data is always current and nothing is duplicated. Queries are only as fast and available as the slowest source, and heavy queries put load on operational systems. Use federation for exploration and light reporting, and batch or CDC pipelines for high-volume or repeated workloads.

6. Routing and Transformation Patterns

These patterns decide where a message goes and what shape it has when it arrives.

Routers choose a path; splitters and scatter-gather fan work out and back in



routing patterns · 3 flows

A router picks one path based on content. A splitter breaks one message into many and an aggregator recombines the results. Scatter-gather sends the same request to several services and merges their replies.

6.1 Content-based router

The router inspects each message and sends it to a destination chosen by its content. **Use case:** express orders go to express fulfilment, standard orders to the normal queue, and anything unrecognized to manual review. The sample is an Apache Camel route.

```
from("jms:queue:orders.in")
    .choice()
        .when().jsonpath("$[?(@.priority == 'express')]")
            .to("jms:queue:fulfilment.express")
        .when().jsonpath("$[?(@.priority == 'standard')]")
            .to("jms:queue:fulfilment.standard")
        .otherwise()
            .to("jms:queue:orders.review")
    .end();
```

Trade-offs: keeps routing rules in one place so senders stay simple. Always keep an `otherwise` path so unknown messages are never lost.

6.2 Splitter and aggregator

A splitter turns one composite message into one message per item, and an aggregator collects the results back into one. **Use case:** a supplier sends an order with many lines, each line is priced in parallel, and one priced confirmation is returned.

```
from concurrent.futures import ThreadPoolExecutor

def split(batch: dict) -> list[dict]:
    return batch["lines"]  # splitter: one item per order line

def price_line(line: dict) -> dict:
    return {**line, "price": lookup_price(line["sku"]) * line["qty"]}  # per-
item work

def process_batch(batch: dict) -> dict:
    with ThreadPoolExecutor(max_workers=8) as pool:
        priced = list(pool.map(price_line, split(batch)))
    return {  # aggregator
        "orderId": batch["orderId"],
        "lines": priced,
        "total": sum(line["price"] for line in priced),
    }
```

Trade-offs: parallel work and smaller units to retry. In message-based flows, carry a correlation id and decide what to do when some items fail or never arrive.

6.3 Scatter-gather

The same request goes to several recipients at once, and a gatherer combines their replies. **Use case:** request quotes from three suppliers and pick the lowest price that arrives within two seconds.

```
import asyncio

async def get_quote(supplier: str, item: str) -> dict:
    ...  # call the supplier's API

async def best_quote(item: str, suppliers: list[str]) -> dict:
    tasks = [asyncio.create_task(get_quote(s, item)) for s in suppliers]
    done, pending = await asyncio.wait(tasks, timeout=2.0)
    for task in pending:
        task.cancel()  # ignore slow suppliers
    quotes = [t.result() for t in done if not t.exception()]
    return min(quotes, key=lambda q: q["price"])  # raises if nobody
answered: handle it
```

Trade-offs: total latency is the slowest accepted reply, not the sum. Define the timeout and what counts as a usable partial result up front.

6.4 Message translator

Converts a message from one format or structure to another so sender and receiver need not share a schema. **Use case:** a legacy ERP emits XML while the new order API expects JSON.

```
import xml.etree.ElementTree as ET

def erp_xml_to_json(xml_text: str) -> dict:
    root = ET.fromstring(xml_text)
    return {
        "orderId": root.findtext("OrderNo"),
        "customerId": root.findtext("Cust/Id"),
        "total": float(root.findtext("Amount")),
    }
```

Trade-offs: isolates format differences in one place. Translators hold business meaning, so test them with real samples and version them with the schemas they convert.

7. Resilience and Transaction Patterns

These patterns keep integrations correct when networks fail, services slow down or messages are delivered twice.

7.1 Retry with backoff

A failed call is repeated after growing delays, with random jitter so many clients do not retry in lockstep. **Use case:** a payment API returns a brief `503` during a deployment. Retry only calls that are safe to repeat, or attach an idempotency key (section 7.4).

```
import random
import time
import requests

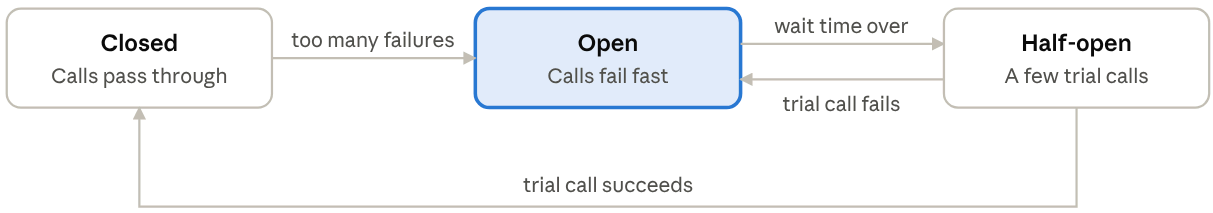
def call_with_retry(url: str, payload: dict, attempts: int = 5):
    for attempt in range(attempts):
        try:
            response = requests.post(url, json=payload, timeout=5)
            if response.status_code < 500:
                return response  # success or a client error: do not retry
        except requests.RequestException:
            pass
        time.sleep(min(30, 2 ** attempt) + random.uniform(0, 1))  # backoff
with jitter
    raise RuntimeError("service still failing after retries")
```

Trade-offs: hides short outages from users. Retrying a service that is truly down adds load, which is why retries pair with a circuit breaker.

7.2 Circuit breaker

After repeated failures the breaker opens and calls fail immediately, giving the struggling service room to recover. After a wait it lets a few trial calls through and closes again if they succeed. **Use case:** a slow inventory service would otherwise tie up every checkout thread.

A breaker opens after repeated failures and probes before it closes again



circuit breaker · 3 states

```
import time

class CircuitBreaker:
    def __init__(self, max_failures=5, reset_after=30):
        self.max_failures, self.reset_after = max_failures, reset_after
        self.failures, self.opened_at = 0, None

    def call(self, fn, *args, **kwargs):
        if self.opened_at is not None:
            if time.time() - self.opened_at < self.reset_after:
                raise RuntimeError("circuit open: failing fast")
            # wait is over: half-open, let this call act as the trial
        try:
            result = fn(*args, **kwargs)
        except Exception:
            self.failures += 1
            if self.failures >= self.max_failures or self.opened_at is not
None:
                self.opened_at = time.time()  # open, or reopen after a
failed trial
            raise
        self.failures, self.opened_at = 0, None  # success closes the circuit
        return result
```

Trade-offs: protects callers and the failing service. Thresholds need tuning, and a fallback (cached value, default, friendly error) decides what users see while it is open. In production, use a maintained library such as resilience4j or Polly.

7.3 Dead-letter queue (DLQ)

Messages that cannot be processed are moved to a separate queue instead of blocking the main one or being lost. **Use case:** one malformed order message keeps failing and would otherwise stall the queue. The sample configures RabbitMQ so rejected messages go to `orders.dlq`.

```
channel.queue_declare(queue="orders.dlq", durable=True)

channel.queue_declare(
    queue="orders",
    durable=True,
    arguments={
        "x-dead-letter-exchange": "",  # default exchange
        "x-dead-letter-routing-key": "orders.dlq",
    },
)

def handle(ch, method, props, body):
    try:
        process_order(body)
        ch.basic_ack(method.delivery_tag)
    except Exception:
        ch.basic_nack(method.delivery_tag, requeue=False)  # routes it to the
DLQ
```

Trade-offs: nothing is silently lost and failures can be inspected and replayed. A DLQ nobody watches is just a hiding place, so alert on its depth.

7.4 Idempotent receiver

Processing the same message twice has the same effect as processing it once. **Use case:** at-least-once delivery redelivers a payment message, and the customer must be charged once. The sample records each message id in the same transaction as the business change (PostgreSQL syntax).

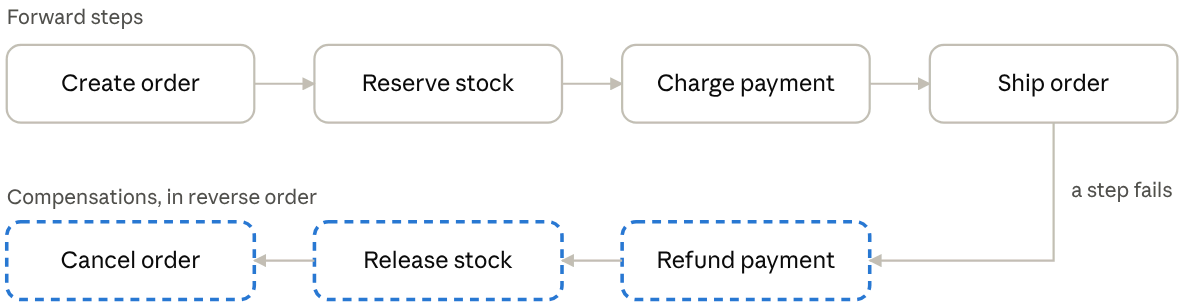
```
def handle_payment(conn, message_id: str, payload: dict) -> None:
    with conn:  # one transaction
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO processed_messages (message_id) VALUES (%s) ON
CONFLICT DO NOTHING",
            (message_id,),
        )
        if cur.rowcount == 0:
            return  # duplicate: already processed
        apply_payment(cur, payload)
```

Trade-offs: makes retries and redelivery safe. Senders must supply a stable unique id, and the id store needs a retention policy.

7.5 Saga (compensating transactions)

A business transaction spanning several services is a sequence of local steps. If one fails, compensating actions undo the earlier steps in reverse order, instead of holding a distributed lock. **Use case:** placing an order creates the order, reserves stock, charges payment and ships.

If a step fails, the saga runs compensations for the steps already done



saga with compensation · 4 steps

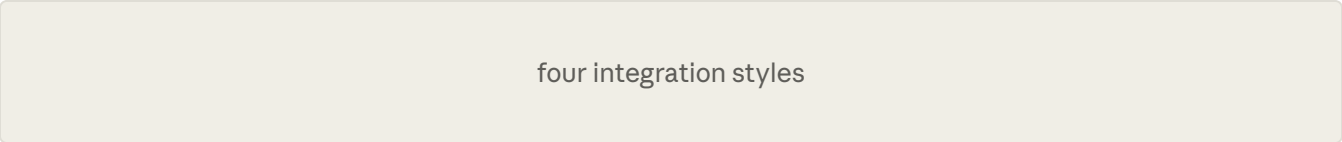
The orchestrator below runs the steps and, on failure, runs the recorded compensations in reverse.

```
def place_order(order):
    steps = [
        (create_order, cancel_order),
        (reserve_stock, release_stock),
        (charge_payment, refund_payment),
        (ship_order, None),
    ]
    done = []  # compensations for steps that completed
    try:
        for action, compensate in steps:
            action(order)
            if compensate:
                done.append(compensate)
    except Exception:
        for compensate in reversed(done):
            compensate(order)
        raise
```

Trade-offs: works across services without two-phase commit, but results are only eventually consistent. Compensations must be idempotent, and some effects (a sent email) cannot be fully undone.

8. Integration Styles

Before picking a pattern, decide how applications will share data at all. Four basic styles exist, and the patterns in sections 3 to 7 mostly build on the last two.



Files and shared databases are the simplest and the most tightly bound. A remote call is immediate but needs both sides running. Messaging decouples them in time.

8.1 File transfer

One application writes a file and another reads it later. **Use case:** a payroll system drops a nightly CSV file that the accounting system imports. The sample writes to a temporary name and renames at the end, so a reader never sees a half-written file.

```
import csv
import os

def export_orders(rows, folder):
    tmp = os.path.join(folder, "orders.csv.tmp")
    with open(tmp, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["order_id", "customer_id", "total"])
        writer.writerows(rows)
    os.replace(tmp, os.path.join(folder, "orders.csv"))  # rename last

def import_orders(path):
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            save_order(row)
```

Trade-offs: simple, works across almost any system and suits bulk data. It is slow to react, error handling is manual, and the file format becomes an unversioned contract.

8.2 Shared database

Several applications read and write the same tables. **Use case:** billing and support both use one customer table. If you must do this, expose a view as a stable contract instead of the raw tables (SQL shown).

```
CREATE VIEW customer_contract_v1 AS
SELECT id AS customer_id, full_name, email
FROM customers;

GRANT SELECT ON customer_contract_v1 TO billing_app;
```

Trade-offs: data is always current and no transport is needed. The schema becomes a contract that every application depends on, so one change can break all of them, and heavy use by one application slows the others.

8.3 Remote call (RPC / API)

The caller invokes an operation on another system and waits for the answer. Section 4.1 shows a REST call with a timeout. A contract-first alternative is gRPC, where the interface is defined in a Protocol Buffers file:

```
syntax = "proto3";

service OrderService {
  rpc GetOrder (GetOrderRequest) returns (Order);
}

message GetOrderRequest { string order_id = 1; }
message Order { string order_id = 1; double total = 2; }
```

Trade-offs: immediate answers and simple reasoning. Both systems must be available at the same time, and each call adds latency and a failure point.

8.4 Messaging

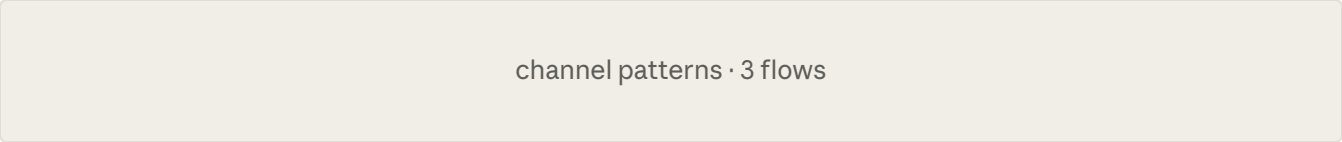
Applications exchange messages through a channel, so sender and receiver need not be available together. Sections 3, 4 and 9 to 12 cover the patterns built on it.

8.5 Comparing the styles

Style	Timing	Coupling	Best for	Main weakness
File transfer	Batch	Low, through the file format	Bulk exchange with legacy or partner systems	Slow to react
Shared database	Immediate	Very high, through the schema	A single team sharing one dataset	Schema changes break everyone
Remote call	Synchronous	Medium, through the API	Questions that need an answer now	Needs both sides available
Messaging	Asynchronous	Low, through the message format	Decoupled, resilient flows	More moving parts to operate

9. Channels and Endpoints

Channels carry messages, and endpoints connect applications to them. The point-to-point and publish/subscribe channels are in sections 3.1 and 3.2, and the dead-letter channel is in section 7.3. This section covers the remaining channel patterns and the endpoint patterns.



A guaranteed-delivery channel stores messages until they are acknowledged, an adapter connects an application that cannot send messages, and a bridge joins two messaging systems.

9.1 Guaranteed delivery

The channel persists each message before confirming it and keeps it until the receiver acknowledges it, so a crash does not lose messages. **Use case:** payment instructions must survive a broker restart. The sample uses RabbitMQ with publisher confirms and a durable queue. The consumer acknowledges only after processing, as in section 3.1.

```
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
channel = connection.channel()
channel.confirm_delivery()  # the broker confirms it has stored each message
channel.queue_declare(queue="payments", durable=True)

channel.basic_publish(
    exchange="",
    routing_key="payments",
    body=body,
    properties=pika.BasicProperties(delivery_mode=2),  # persistent
    mandatory=True,
)
```

Trade-offs: no loss on failure, at the cost of speed from disk writes and confirmations. Delivery may repeat, so consumers must be idempotent (section 7.4).

9.2 Channel adapter

An adapter connects an application that does not speak messaging, such as a database, a file system or a SaaS API, to a channel. **Use case:** a legacy application writes rows to a table, and the adapter turns each new row into a message.

```
def database_adapter(conn, publish):
    cur = conn.cursor()
    cur.execute("SELECT id, payload FROM legacy_events WHERE sent = false
ORDER BY id")
    for row_id, payload in cur.fetchall():
        publish("legacy.events", payload)
        cur.execute("UPDATE legacy_events SET sent = true WHERE id = %s",
(row_id,))
    conn.commit()
```

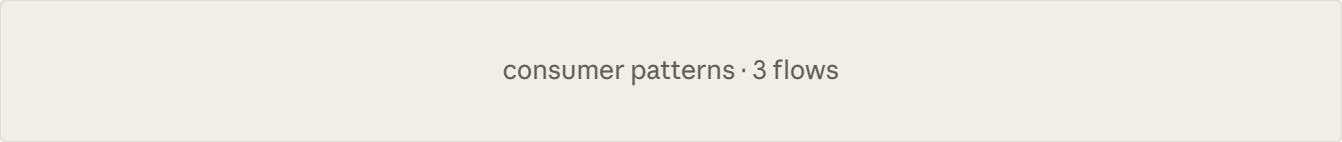
Trade-offs: adds messaging to an application you cannot change. The adapter is another component to run, and it needs its own error handling and duplicate protection.

9.3 Messaging bridge

A bridge forwards messages between two messaging systems so applications on each side can communicate. **Use case:** a legacy JMS broker feeds a new Kafka platform. The Apache Camel route below copies each message.

```
from("jms:queue:orders.legacy")
    .to("kafka:orders?brokers=kafka.example.com:9092");
```

Trade-offs: lets two worlds interoperate during a migration. Each bridge is a place where delivery guarantees, ordering and message formats can be weakened, so test it under failure.



A consumer can poll for work, be pushed each message, or share a queue with other consumers.

9.4 Polling consumer

The consumer asks the channel for messages on its own schedule. **Use case:** a worker reads an Amazon SQS queue and waits when it is empty. Long polling avoids a busy loop.

```
while True:
    response = sqs.receive_message(
        QueueUrl=queue_url, MaxNumberOfMessages=10, WaitTimeSeconds=20  #
long poll
    )
    for message in response.get("Messages", []):
        handle(message["Body"])
        sqs.delete_message(QueueUrl=queue_url,
ReceiptHandle=message["ReceiptHandle"])
```

Trade-offs: the consumer controls its own pace and load. It adds polling delay and wasted calls when the queue is quiet.

9.5 Event-driven consumer

The channel or platform pushes each message to the handler as it arrives. **Use case:** an AWS Lambda function triggered by an SQS queue.

```
def lambda_handler(event, context):
    for record in event["Records"]:
        handle(record["body"])  # the platform delivers each message to this
function
```

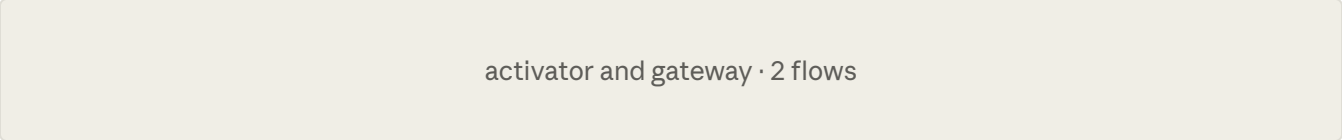
Trade-offs: low latency and no polling code. The platform decides the delivery rate, so protect downstream systems with concurrency limits.

9.6 Competing consumers

Several consumers read the same queue, and each message goes to exactly one of them. **Use case:** order processing spreads across several worker instances. Start the same worker more than once.

```
# Run this worker several times; the broker gives each message to one of them
channel.basic_qos(prefetch_count=1)  # fair dispatch: one unacknowledged
message at a time
channel.basic_consume(queue="orders", on_message_callback=handle)
channel.start_consuming()
```

Trade-offs: throughput scales by adding workers. Order across workers is not guaranteed, so use a single consumer, or route related messages to one consumer, where order matters.



An activator turns a message into a service call, and a gateway hides the messaging API from application code.

9.7 Service activator

An activator receives a message and invokes the matching operation on a service, so the service itself knows nothing about messaging. **Use case:** a `ConfirmOrder` message triggers the order service's confirm operation.

```
def activate(message: dict, order_service) -> None:
    if message["type"] == "ConfirmOrder":
        order_service.confirm(message["orderId"])
```

Trade-offs: keeps business logic free of broker code and easy to test. The mapping from message to call must be maintained as messages evolve.

9.8 Messaging gateway

A gateway wraps the messaging API in a small interface, so application code never touches the broker library. **Use case:** business code calls `place_order` and does not know whether a queue, a topic or an HTTP call sits behind it.

```
import json

class OrderGateway:
    def __init__(self, channel):
        self._channel = channel

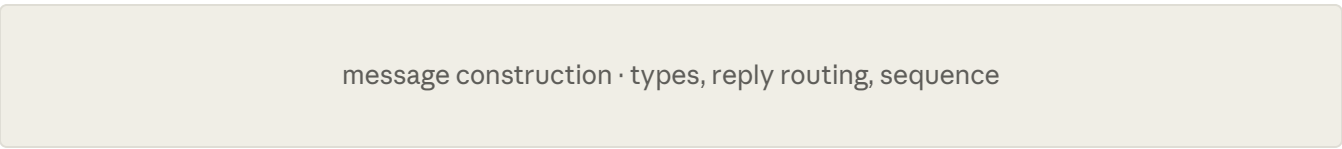
    def place_order(self, order: dict) -> None:
        self._channel.basic_publish(
            exchange="", routing_key="orders", body=json.dumps(order)
        )

gateway = OrderGateway(channel)
gateway.place_order({"orderId": "A-1001", "total": 59.9})  # no broker API in
business code
```

Trade-offs: easy to swap the transport and to replace with a fake in tests. One more layer to maintain.

10. Message Construction

A message needs more than a payload. Its type says what the receiver should do with it, and its headers say how replies find their way back and how parts fit together.



The top row shows the three message types. The middle row shows a reply finding its requester, and the bottom row shows a large payload split into numbered parts.

10.1 Command, document and event messages

Type	Intent	Receivers	Naming	Example
Command	Do this	One intended handler	Imperative	`ConfirmOrder`

Type	Intent	Receivers	Naming	Example
Document	Here is data to use	Whoever needs the data	Noun	`OrderDocument`
Event	This happened	Any interested party	Past tense	`OrderConfirmed`

```
command = {"type": "ConfirmOrder", "orderId": "A-1001"}  # do this
document = {"type": "OrderDocument", "orderId": "A-1001", "lines": []}  #
here is data
event = {"type": "OrderConfirmed", "orderId": "A-1001", "at": "2026-10-
13T14:05:00Z"}  # it happened
```

Use case: a command goes to the one service that owns confirmation. The confirmed event goes to everyone who cares, and the order document is sent to a partner who needs the full data.

Trade-offs: being clear about the type keeps coupling honest. A command couples the sender to a specific handler, while an event does not.

10.2 Correlation ID and return address

In request/reply over messaging, the request carries a **return address** (the queue to answer on) and a **correlation ID** (an identifier the reply repeats), so the requester can match each reply to its request. **Use case:** a pricing service answers many requesters through one request queue. The sample uses RabbitMQ with `pika`; the requester publishes first.

```
import json
import uuid
import pika

result = channel.queue_declare(queue="", exclusive=True)  # private reply
queue
reply_queue = result.method.queue
correlation_id = str(uuid.uuid4())

channel.basic_publish(
    exchange="",
    routing_key="price-requests",
    properties=pika.BasicProperties(reply_to=reply_queue,
correlation_id=correlation_id),
    body=json.dumps({"sku": "X-7"}),
)
# When a reply arrives, accept it only if props.correlation_id ==
correlation_id
```

The service answers on the address it was given and echoes the ID:

```
def on_request(ch, method, props, body):
    price = lookup_price(json.loads(body)["sku"])
    ch.basic_publish(
        exchange="",
        routing_key=props.reply_to,  # return address
        properties=pika.BasicProperties(correlation_id=props.correlation_id),
# echo the ID
        body=json.dumps({"price": price}),
    )
    ch.basic_ack(method.delivery_tag)
```

Trade-offs: lets many requesters share one service without mixing up replies. The requester must handle replies that never arrive, so set a timeout.

10.3 Message sequence

When a payload is too large for one message, split it into numbered parts that carry the same message id and the total count, and reassemble at the end. **Use case:** a large export is sent through a broker with a message size limit.

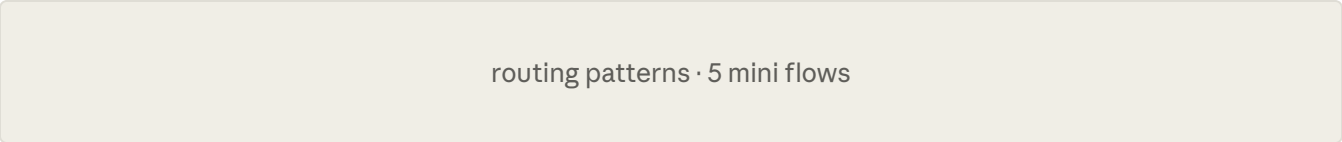
```
def split_message(data: bytes, size: int, message_id: str) -> list:
    parts = [data[i:i + size] for i in range(0, len(data), size)]
    return [
        {"messageId": message_id, "index": i + 1, "total": len(parts),
"body": part}
        for i, part in enumerate(parts)
    ]

def reassemble(parts: list) -> bytes:
    ordered = sorted(parts, key=lambda p: p["index"])
    if len(ordered) != ordered[0]["total"]:
        raise ValueError("parts missing")
    return b"".join(p["body"] for p in ordered)
```

Trade-offs: works around size limits, but the receiver must collect and order parts and handle missing ones. For large payloads, the claim check in section 11.8 is usually simpler.

11. Further Routing and Transformation Patterns

Section 6 covered the content-based router, splitter, aggregator, scatter-gather and message translator. These patterns round out routing and transformation.



A filter drops messages, a recipient list chooses several destinations, a resequencer fixes order, a routing slip puts the route in the message, and a process manager keeps the plan in one place.

11.1 Message filter

A filter drops messages the receiver does not need. **Use case:** the EU fulfilment system only wants `OrderPlaced` events for EU customers.

```
def should_forward(message: dict) -> bool:
    return message["type"] == "OrderPlaced" and message["region"] == "EU"

def filter_messages(messages, forward):
    for message in messages:
        if should_forward(message):
            forward(message)  # everything else is dropped
```

Trade-offs: less load and simpler receivers. Dropped messages are invisible, so count them and log a sample.

11.2 Recipient list

The router computes a list of recipients for each message and sends a copy to each. Unlike a content-based router, one message can go to several places. **Use case:** every order goes to audit, to the regional fulfilment queue, and to fraud review when it is large.

```
def recipients(order: dict) -> list:
    targets = ["audit"]
    targets.append("fulfilment-eu" if order["region"] == "EU" else
"fulfilment-us")
    if order["total"] > 10_000:
        targets.append("fraud-review")
    return targets
```

The same idea in Apache Camel:

```
from("direct:orders")

.recipientList(simple("jms:queue:fulfilment.${header.region},jms:queue:audit"
));
```

Trade-offs: flexible fan-out controlled by rules. The sender is coupled to the list logic, so keep the rules in one place and test them.

11.3 Resequencer

A resequencer holds messages that arrive out of order and releases them in order. **Use case:** parts of a document arrive from parallel workers, but the target needs them in sequence.

```
import heapq

class Resequencer:
    def __init__(self, first_index=1):
        self.next_index = first_index
        self.waiting = []  # min-heap of (index, tie-breaker, message)

    def accept(self, index: int, message: dict) -> list:
        heapq.heappush(self.waiting, (index, id(message), message))
        released = []
        while self.waiting and self.waiting[0][0] == self.next_index:
            released.append(heapq.heappop(self.waiting)[2])
            self.next_index += 1
        return released
```

Trade-offs: restores order for consumers that need it. A missing message blocks everything behind it, so add a timeout and a policy for gaps.

11.4 Routing slip

The message carries the list of steps it must visit. Each step does its work and forwards the message to the next destination on the slip, so no central controller is needed. **Use case:** an invoice visits different steps depending on its type, such as validation, enrichment and archiving.

```
def run_step(message: dict, do_work, send) -> None:
    message["data"] = do_work(message["data"])
    if message["slip"]:
        next_queue = message["slip"].pop(0)  # the route travels with the
message
        send(next_queue, message)
```

Trade-offs: flexible per-message routes and no central bottleneck. The overall flow is harder to monitor, and the slip format becomes a contract between steps.

11.5 Process manager

A central process manager keeps the state of a multi-step process and decides the next step from each reply. It is the orchestration alternative to a routing slip, and the saga in section 7.5 is a special case that adds compensation. **Use case:** a loan application whose next step depends on the results of earlier checks.

```
def next_step(state: dict, reply: dict) -> str:
    if state["step"] == "validate":
        return "enrich" if reply["valid"] else "reject"
    if state["step"] == "enrich":
        return "approve" if reply["total"] > 1000 else "archive"
    return "done"
```

Trade-offs: the flow is visible and easy to change in one place. The manager is a dependency that must be reliable and can grow into a bottleneck.

transformation patterns · 5 mini flows

An enricher adds data, a content filter removes fields, a claim check parks the payload, an envelope carries headers, and a normalizer unifies formats.

11.6 Content enricher

The enricher adds data the message lacks by looking it up in another source. **Use case:** an order carries only a customer id, and the receiver needs the customer's name and tier.

```
def enrich_order(order: dict, customer_service) -> dict:
    customer = customer_service.get(order["customerId"])
    return {**order, "customerName": customer["name"], "tier":
customer["tier"]}
```

Trade-offs: receivers get complete messages. Each lookup adds latency and a dependency, so cache results where the data changes slowly.

11.7 Content filter

A content filter removes fields from a message and passes on a smaller one. This differs from the message filter, which drops whole messages. **Use case:** order events go to analytics without personal data.

```
def slim_order(order: dict) -> dict:
    allowed = ("orderId", "customerId", "total")
    return {key: order[key] for key in allowed}
```

Trade-offs: smaller messages and less exposure of sensitive data. Use an allow-list as shown, so a new field is excluded by default.

11.8 Claim check

The large payload is stored aside and the message carries only a reference. The receiver redeems the reference to fetch the payload. **Use case:** scanned documents are too big for the broker's message limit. The sample uses Amazon S3.

```
import uuid
import boto3

s3 = boto3.client("s3")

def check_in(payload: bytes, bucket: str) -> dict:
    key = f"claims/{uuid.uuid4()}"
    s3.put_object(Bucket=bucket, Key=key, Body=payload)
    return {"claimCheck": {"bucket": bucket, "key": key}}  # small message
for the broker

def check_out(message: dict) -> bytes:
    claim = message["claimCheck"]
    return s3.get_object(Bucket=claim["bucket"], Key=claim["key"])
["Body"].read()
```

Trade-offs: keeps messages small and the broker fast. The store needs access control and a clean-up rule for payloads that are no longer needed.

11.9 Envelope wrapper

The sender wraps the message in an envelope with the headers a transport needs, such as destination, trace id and content type. The receiver unwraps it before use. **Use case:** a gateway to a partner requires routing and tracing headers that the business payload should not carry.

```
def wrap(body: dict, destination: str, trace_id: str) -> dict:
    return {
        "headers": {"to": destination, "traceId": trace_id, "contentType":
"application/json"},
        "body": body,
    }

def unwrap(envelope: dict) -> dict:
    return envelope["body"]
```

Trade-offs: keeps business data and transport data apart. Both ends must agree on the envelope format.

11.10 Normalizer

A normalizer accepts messages in several formats and converts each to one standard shape. **Use case:** partners send orders as JSON, XML or CSV, and the rest of the system expects one format. It pairs naturally with the canonical data model in section 5.4.

```
import csv
import io
import json
import xml.etree.ElementTree as ET

def from_json(text: str) -> dict:
    data = json.loads(text)
    return {"orderId": data["id"], "total": data["amount"]}

def from_xml(text: str) -> dict:
    root = ET.fromstring(text)
    return {"orderId": root.findtext("Id"), "total":
float(root.findtext("Amount"))}

def from_csv(text: str) -> dict:
    row = next(csv.DictReader(io.StringIO(text)))
    return {"orderId": row["id"], "total": float(row["amount"])}

PARSERS = {"application/json": from_json, "application/xml": from_xml,
"text/csv": from_csv}

def normalize(content_type: str, body: str) -> dict:
    return PARSERS[content_type](body)
```

Trade-offs: downstream code handles one format. Each new source format needs a parser, and unknown formats must be rejected or sent to a dead-letter queue.

12. Legacy Migration and Monitoring Patterns

These patterns help you replace a system without a big-bang cutover and observe message flows without disturbing them.

migration and monitoring patterns · 3 flows

A proxy lets you move features gradually, a translator protects the new design from the old model, and a tap copies traffic for audit or debugging.

12.1 Strangler fig

Place a proxy in front of the legacy system and move one feature at a time to the new one, until the legacy system handles nothing and can be retired. **Use case:** migrate order handling first, then billing, then reporting. The NGINX configuration below sends migrated routes to the new system and everything else to the old one.

```
upstream legacy_app { server legacy.internal:8080; }
upstream new_app    { server new.internal:8080; }

server {
    listen 80;

    # Migrated features go to the new system
    location /api/orders/ { proxy_pass http://new_app; }

    # Everything else still goes to the legacy system
    location / { proxy_pass http://legacy_app; }
}
```

Trade-offs: low-risk, reversible steps and early value. Data must stay consistent between old and new during the transition (sections 5.2 and 5.3 help), and the proxy becomes critical infrastructure.

12.2 Anti-corruption layer

A translation layer converts the legacy system's model into your own, so its odd field names and codes do not leak into the new design. **Use case:** the new customer service reads a legacy customer master table with cryptic columns.

```
class LegacyCustomerAdapter:
    """Anti-corruption layer: translates the legacy model into ours."""

    STATUS_MAP = {"A": "ACTIVE", "I": "INACTIVE", "X": "CLOSED"}

    def __init__(self, legacy_client):
        self.legacy = legacy_client

    def get_customer(self, customer_id: str) -> dict:
        row = self.legacy.fetch("CUST_MASTER", customer_id)  # legacy field
names
        return {
            "customerId": row["CUST_NO"],
            "name": f"{row['FNAME']} {row['LNAME']}".strip(),
            "status": self.STATUS_MAP.get(row["STAT_CD"], "UNKNOWN"),
        }
```

Trade-offs: the new design stays clean and the legacy system can be replaced behind the layer. The layer is extra code to maintain, and its mappings need tests against real legacy data.

12.3 Wire tap

A tap sends a copy of each message to a side channel, such as an audit log or a debugging queue, while the original continues unchanged. **Use case:** keep an audit trail of every order message without slowing processing. The Apache Camel route below copies each message to an audit queue.

```
from("jms:queue:orders.in")
    .wireTap("jms:queue:orders.audit")
    .to("direct:process");
```

Trade-offs: adds visibility with little effect on the main flow. Copies may contain sensitive data, so apply the same access control and retention rules to the audit channel, or filter fields with a content filter (section 11.7) first.

13. Mapping the Patterns to Boomi

Boomi implements most of these patterns with shapes, connectors and runtimes rather than hand-written code. The table is a guide to the usual building blocks. Component names change between releases, so check the current Boomi documentation before you

rely on one.

Pattern	Typical Boomi building blocks
Point-to-point	One process: source connector, Map shape, target connector
Hub-and-spoke / ESB	Atoms and Molecules hosting many processes, with shared connections and subprocesses
API-led connectivity	API Management (API proxies and components) in front of layered processes
Message queue	Atom Queue connector, or JMS, Amazon SQS and Azure Service Bus connectors
Publish/subscribe, streaming	Boomi Event Streams, or the Kafka and JMS topic connectors
Request/reply	Web Services Server listener returning a response document; HTTP Client for outbound calls
Fire-and-forget	Listener process that acknowledges at once and hands the work to a queue or separate process
Asynchronous request-reply	Listener returns a job id; a separate process does the work; a second listener returns status
Batch/ETL	Scheduled process with database or file connectors, Map and Data Process shapes
Change data capture	Database Get with a last-run watermark, or log-based CDC through Boomi Data Integration (verify connector support)
Data synchronization	Boomi DataHub for matching and golden records, or a process with conflict rules
Canonical data model	Shared canonical profile and Map components reused across processes
Content-based router	Decision and Route shapes
Splitter / aggregator	Data Process (split and combine documents), Flow Control for parallelism

Pattern	Typical Boomi building blocks
Scatter-gather	Parallel Process Calls or Flow Control, then combine documents (verify parallel behavior for your runtime)
Message translator	Map shape, profile conversions, Data Process scripting
Retry	Try/Catch shape with a retry count, or a reusable error-handling subprocess
Circuit breaker	No dedicated shape: keep failure state in a cache or process property and test it with a Decision before the call
Dead-letter queue	Catch path that writes failed documents to a queue, table or file and raises a notification
Idempotent receiver	Document cache or database lookup keyed on message id, tested with a Decision
Saga	Orchestrating process with Try/Catch and compensating subprocesses called via Process Call

Package the cross-cutting patterns (retry, logging, dead-letter handling, duplicate checks) as reusable subprocesses in a shared library. Every integration then inherits the same resilience behavior instead of rebuilding it.

Newer patterns in Boomi

The patterns added in sections 8 to 12 map to Boomi as follows. Items marked "verify" depend on your Boomi release and runtime, so check the current documentation.

Pattern	Typical Boomi building blocks
File transfer	Disk, FTP and SFTP connectors, started by a schedule or a listener
Shared database	Database connector against agreed views or tables

Pattern	Typical Boomi building blocks
Guaranteed delivery	Persistent queue connectors (Atom Queue, JMS) with acknowledgement after processing
Channel adapter	Database, Disk and application connectors that turn external data into documents
Messaging bridge	A process that reads from one broker connector and writes to another
Polling consumer	Scheduled process using a queue or database Get operation
Event-driven consumer	Listener (subscribe) operation that starts the process on arrival
Competing consumers	Several runtime nodes or concurrent executions reading one queue (verify)
Service activator	Listener process that calls a connector or a subprocess
Messaging gateway	A reusable subprocess that hides the send operation
Command, document and event messages	One profile per message type, with document properties for headers
Correlation ID and return address	Dynamic document properties or message headers, plus the listener's response
Message sequence	Data Process split and combine, with a sequence number in a document property
Message filter	Decision shape that stops or drops documents
Recipient list	Decision and Branch shapes, or a Route shape driven by a computed value
Resequencer	Sort in a Data Process step, or a cache read in order (verify)
Routing slip	Route shape driven by a document property listing the steps (verify)

Pattern	Typical Boomi building blocks
Process manager	Orchestrating process with Process Call, Decision and Try/Catch shapes
Content enricher	Connector or cache lookup, with a Map to merge the result
Content filter	Map shape that carries only the fields needed
Claim check	Store the payload with a Disk or cloud storage connector and pass a reference
Envelope wrapper	Map or message-building step that adds headers, and a Map that removes them
Normalizer	One profile and Map per input format, all producing the canonical profile
Strangler fig	API Management or a gateway routing between old and new endpoints
Anti-corruption layer	Process with Maps from the legacy profile to your canonical profile
Wire tap	Branch shape that copies documents to a logging or audit path
Data federation	No native shape; use a data virtualization or integration product (verify)

14. Pattern Selection Guide

Start from the situation, pick the pattern it points to, then add resilience patterns for every remote call.

Situation	Pattern	Section
Two stable systems, low change	Point-to-point	2.1
Many systems sharing rules and mappings	Hub-and-spoke or ESB	2.2, 2.3
Reusable capabilities for several channels	API-led connectivity	2.4

Situation	Pattern	Section
Caller needs an answer now	Request/reply with a timeout	4.1
Work takes minutes	Asynchronous request-reply	4.3
Traffic spikes, receiver is slower	Message queue	3.1
Many consumers react to one event	Publish/subscribe or streaming	3.2, 3.3
Large volumes moved on a schedule	Batch/ETL	5.1
Changes needed within seconds, deletes included	Change data capture	5.2
Same record edited in several systems	Data synchronization	5.3
Many formats across many systems	Canonical data model	5.4
Destination depends on content	Content-based router	6.1
One request to several providers	Scatter-gather	6.3
Downstream service is unreliable	Retry plus circuit breaker	7.1, 7.2
Messages that keep failing	Dead-letter queue	7.3
Messages may arrive twice	Idempotent receiver	7.4
Business transaction spans services	Saga	7.5

More situations

Situation	Pattern	Section
Choosing how applications share data at all	Integration styles	8
Messages must survive a crash	Guaranteed delivery	9.1
An application cannot send or receive messages	Channel adapter	9.2
Two messaging systems must exchange messages	Messaging bridge	9.3
Scale out workers on one queue	Competing consumers	9.6

Situation	Pattern	Section
Keep broker code out of business logic	Service activator, messaging gateway	9.7, 9.8
Match replies to their requests	Correlation ID and return address	10.2
A payload is too large to send in one message	Message sequence or claim check	10.3, 11.8
A receiver should get only some messages	Message filter	11.1
One message must reach several computed destinations	Recipient list	11.2
Messages arrive out of order	Resequencer	11.3
The route differs for each message	Routing slip	11.4
One owner for a multi-step process	Process manager	11.5
A message lacks data the receiver needs	Content enricher	11.6
A receiver must not see some fields	Content filter	11.7
The transport needs extra headers	Envelope wrapper	11.9
Many input formats must become one	Normalizer	11.10
Replace a legacy system gradually	Strangler fig, anti-corruption layer	12.1, 12.2
Audit messages without disturbing the flow	Wire tap	12.3
Query several sources without copying data	Data federation	5.5

Common anti-patterns

- **Chatty synchronous chains:** A calls B calls C, so latency and failure odds multiply. Break the chain with a queue or an event.
- **Retries without idempotency:** a retried payment can charge twice.
- **A canonical model that tries to model everything:** it becomes a bottleneck. Keep it to shared, stable concepts.

- **A shared database as the integration layer:** every schema change breaks every consumer.
- **No dead-letter path or monitoring:** failures stay invisible until a customer reports them.
- **Unversioned message schemas:** a harmless field change breaks consumers you did not know existed.

Design review checklist

- [ ] Is each interaction synchronous or asynchronous on purpose?
- [ ] Does every remote call have a timeout, bounded retries and a circuit breaker?
- [ ] Is every consumer safe to run twice on the same message?
- [ ] Is there a dead-letter path with an alert on its depth?
- [ ] Does a correlation id follow each request across all systems?
- [ ] Are message and API schemas versioned?
- [ ] Are credentials, transport and access protected (TLS, OAuth 2.0, least privilege)?
- [ ] Does each interface have a named owner?